

Jour 1 : Environnement Bare-Metal en Rust

Rust a deux versions :

Une version **Stable** utilisé pour le dev normal et une version **Nightly** qui est celle qui nous intéresse car **obligatoire pour l'OS dev**. En effet il contient des features dont on a besoin comme

- `#![no_std]` → pas de bibliothèque standard (elle suppose qu'un OS existe déjà ! ce qui n'est pas notre cas car on commence à partir de rien).
- `#![no_main]` → pas de `main()` classique, On gères nous-même le point d'entrée.
- `Inline assembly (asm!)` pour parler directement au CPU

```
# Étape 1 : dépendances système
sudo apt update && sudo apt install -y curl build-essential gcc lld nasm qemu-system-x86 git

# Étape 2 : installer Rust
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
# → choisis option 1 (default)

# Étape 3 : recharger le PATH
source $HOME/.cargo/env

# Étape 4 : passer en nightly
rustup default nightly

# Étape 5 : target bare-metal
rustup target add x86_64-unknown-none

# Étape 6 : cargo-xbuild
cargo install cargo-xbuild

# Étape 7 : vérifier
rustc --version && cargo --version
```

⚠ Note cyber : `curl ... | sh` est pratique mais c'est un vecteur d'attaque si quelqu'un compromise `sh.rustup.rs`, il exécute du code sur la machine.

```
apt install      → outils système (gcc, qemu, lld...)
  ↓
curl | sh        → installe rustup (gestionnaire Rust)
  ↓
source .cargo/env → rend rustc/cargo disponibles dans le terminal
  ↓
rustup nightly   → active les features expérimentales nécessaires
  ↓
target x86_64-none → Rust sait compiler "dans le vide" (sans OS)
  ↓
cargo-xbuild     → recompile les libs de base pour le bare-metal
  ↓
☑ Environnement prêt
```

L'environnement étant installé, on va faire notre premier test avec Rust :

```
(root@kali)-[/home/kali/Day1]
# cargo new hello_world

Creating binary (application) 'hello_world' package
note: see more `Cargo.toml` keys and their definitions at https://doc.rust-lang.org/cargo/reference/manifest.html

(root@kali)-[/home/kali/Day1]
# ls
hello_world

(root@kali)-[/home/kali/Day1]
# cd hello_world

(root@kali)-[/home/kali/Day1/hello_world]
# nano src/main.rs

(root@kali)-[/home/kali/Day1/hello_world]
#

(root@kali)-[/home/kali/Day1/hello_world]
# cargo run
Compiling hello_world v0.1.0 (/home/kali/Day1/hello_world)
Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.21s
Running `target/debug/hello_world`
Project OS with Rust : Day1

(root@kali)-[/home/kali/Day1/hello_world]
#
```

Focus Cyber

Où un attaquant peut frapper dans la chaîne de compilation ?

Étape	Vecteur d'attaque	Exemple réel
Dépendances (crates)	Crate malveillante sur crates.io	SolarWinds-style supply chain
Le compilateur lui-même	Compiler un compilateur backdooré	Ken Thompson "Trusting Trust" (1984)

Le linker script	Modifier la disposition mémoire	Écrase des sections critiques
Les macros procédurales	Injection de code à la compilation	La macro s'exécute chez toi
Variables d'env de build	build.rs malveillant	Exécution arbitraire au build

L'attaque mythique : "Trusting Trust"

Ken Thompson (créateur d'Unix) a démontré en 1984 qu'on peut backdoorer un compilateur de façon **indétectable dans le code source**. Le compilateur infecté injecte le backdoor dans les binaires qu'il produit, y compris les nouvelles versions de lui-même.